SIMATS ENGINEERING

Engineer to Excel

SIMATS

Saveetha Institute of Medical And Technical Sciences

(Declared as Deemed to be University under Section 3 of UGC Act 1956)

CO1 - ASSESSMENT TOOL 2

COURSE NAME: DESIGN AND ANALYSIS OF ALGORITHMS

COURSE CODE: CSA06

COURSE OUTCOME COVERED

CO1: Determine algorithm efficiency using asymptotic notations and mathematical techniques including Master Theorem and substitution method.

(BL-1, BL-2, BL-3)

Assessment Tool Weightage:

QUESTIONS: 10 **Total Marks: 100**

Annexure A

Concept Mapping

Code of Conduct:

I N. Pujitha Naga Sai Lakshmi | 192572119

(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: N. Pujitha

N. Pujitha
192572119

1. Consider the concept map: Algorithm → Input Size (n) → Time Complexity → Asymptotic Notation → Efficiency. Explain the directional relationship between each node and show how an algorithm is evaluated as the input size increases. Extend the map by adding the branches Big-O, Big-Theta, and Big-Omega under Asymptotic Notation. Justify how these notations help classify the growth of running time for large values of n. Interpret how the completed map supports the measurement of algorithm efficiency in a structured way.

2. Consider the concept map: Algorithm → Best-case, Average-case, Worst-case → Time Complexity → Performance Evaluation. Map the relationships among the three cases and explain how they provide different views of algorithm behavior. Extend the map by linking each case to Asymptotic Notation so that the complexity can be expressed formally. Justify why all three cases are important in performance analysis rather than relying on only one measure. Interpret how the overall structure helps compare the practical and theoretical behavior of algorithms.

3. Consider the concept map: Divide and Conquer → Recurrence Relation → $T(n) = aT(n/b) + f(n)$ → Master Theorem. Explain how each node leads to the next in the analysis of recursive algorithms. Extend the map by adding the three cases of the Master Theorem as child nodes and show how each case depends on the comparison between f(n) and $n^{\log_b a}$. Justify how this mapping simplifies the process of deriving time complexity. Interpret how the completed concept map helps students analyze divide-and-conquer algorithms more efficiently.

4. Consider the concept map: Recurrence Relation → Substitution Method → Expansion → Guess → Induction → Verification → Complexity Derivation. Trace the logical flow from a recurrence relation to its final complexity using the substitution method. Add suitable directional links to show how guessing and induction support the proof of the derived complexity. Justify the role of verification in confirming the correctness of the assumed solution. Interpret how this map provides a stepwise framework for solving recurrence relations systematically.

5. Consider the concept map: Recurrence Relation → Recursion Tree → Level Cost → Height of Tree → Total Cost → Time Complexity. Explain

how a recurrence can be represented using a recursion tree and how cost is distributed across levels. Extend the map by including branching factor as a supporting node connected to the tree structure. Justify how summing the level-wise costs leads to the final total cost. Interpret how this map makes recursive complexity analysis more visual and easier to understand.

6. Consider the concept map: Algorithm → Operation Count → Growth Function → Big-O Notation → Upper Bound. Map the conceptual links showing how counting basic operations leads to a growth function and then to an asymptotic upper bound. Extend the map by connecting Worst-case Analysis as a supporting node to Big-O Notation. Justify why Big-O is used to represent an upper limit on running time rather than an exact value. Interpret how the completed structure supports reliable performance estimation for large inputs.

7. Consider the concept map: Algorithm → Growth Function → Big-Omega Notation → Lower Bound → Best-case Analysis. Explain the relationships among the nodes and show how lower-bound analysis captures the minimum behavior of an algorithm. Extend the map by linking Performance Guarantee and Input Characteristics as additional supporting nodes. Justify why Big-Omega is useful when discussing guaranteed minimum efficiency. Interpret how the concept map gives a balanced understanding of algorithm behavior from the lower-bound perspective.

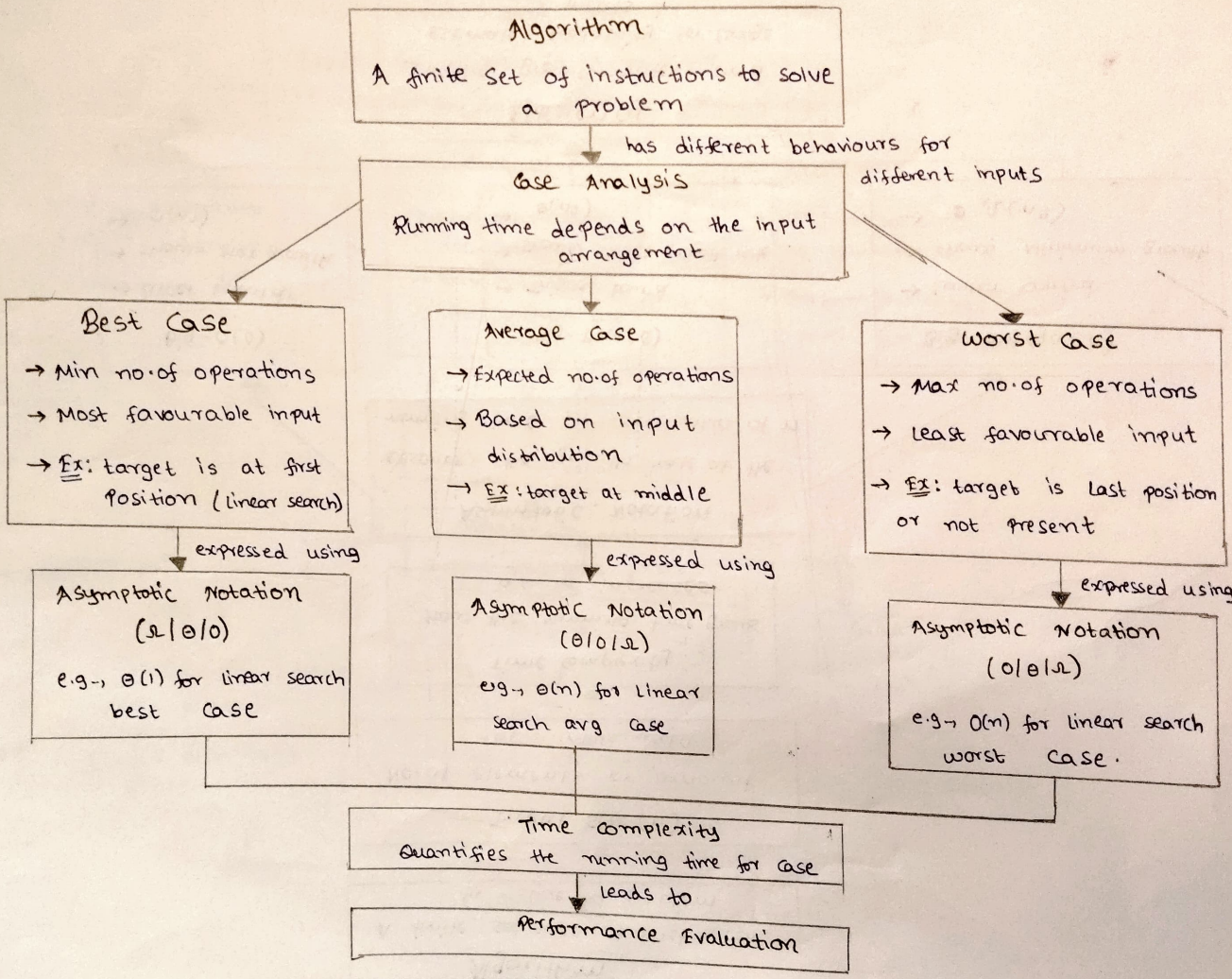
8. Consider the concept map: Algorithm → Growth Function → Big-Theta Notation → Tight Bound → Exact Order of Growth. Explain how the concept of tight bound emerges when upper and lower bounds meet. Extend the map by adding links from Big-O and Big-Omega toward Big-Theta to show their relationship. Justify why Big-Theta is often preferred when the exact asymptotic growth is known. Interpret how the map helps distinguish between approximate and precise complexity descriptions.

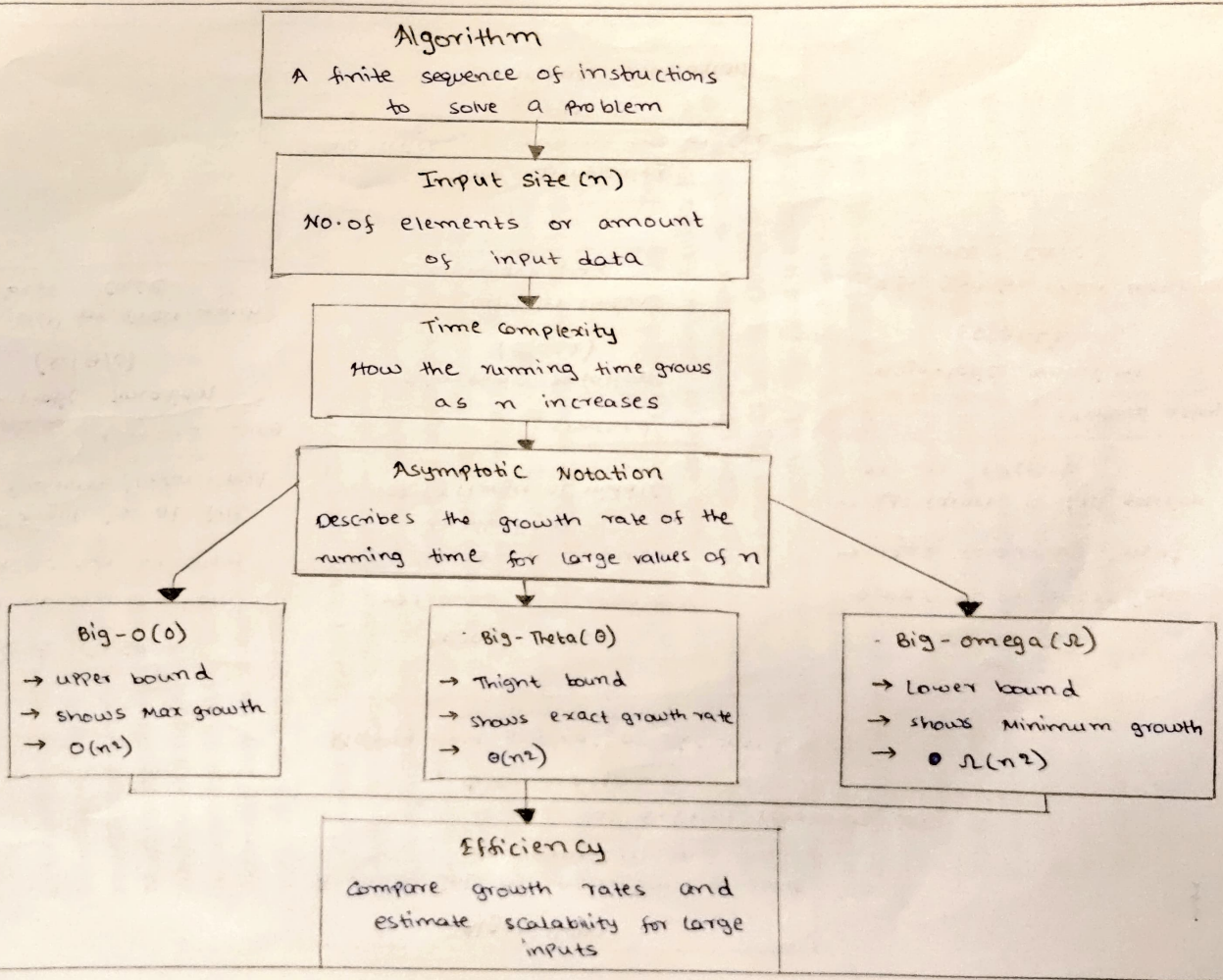
9. Consider the concept map: Algorithm → Input Size → Constant Time, Linear Time, Logarithmic Time, Quadratic Time → Efficiency Classification. Explain how changes in input size lead to different growth patterns and how these patterns are classified. Extend the map by linking Asymptotic Notation to each complexity class. Justify why logarithmic and linear algorithms are generally more scalable than quadratic algorithms.

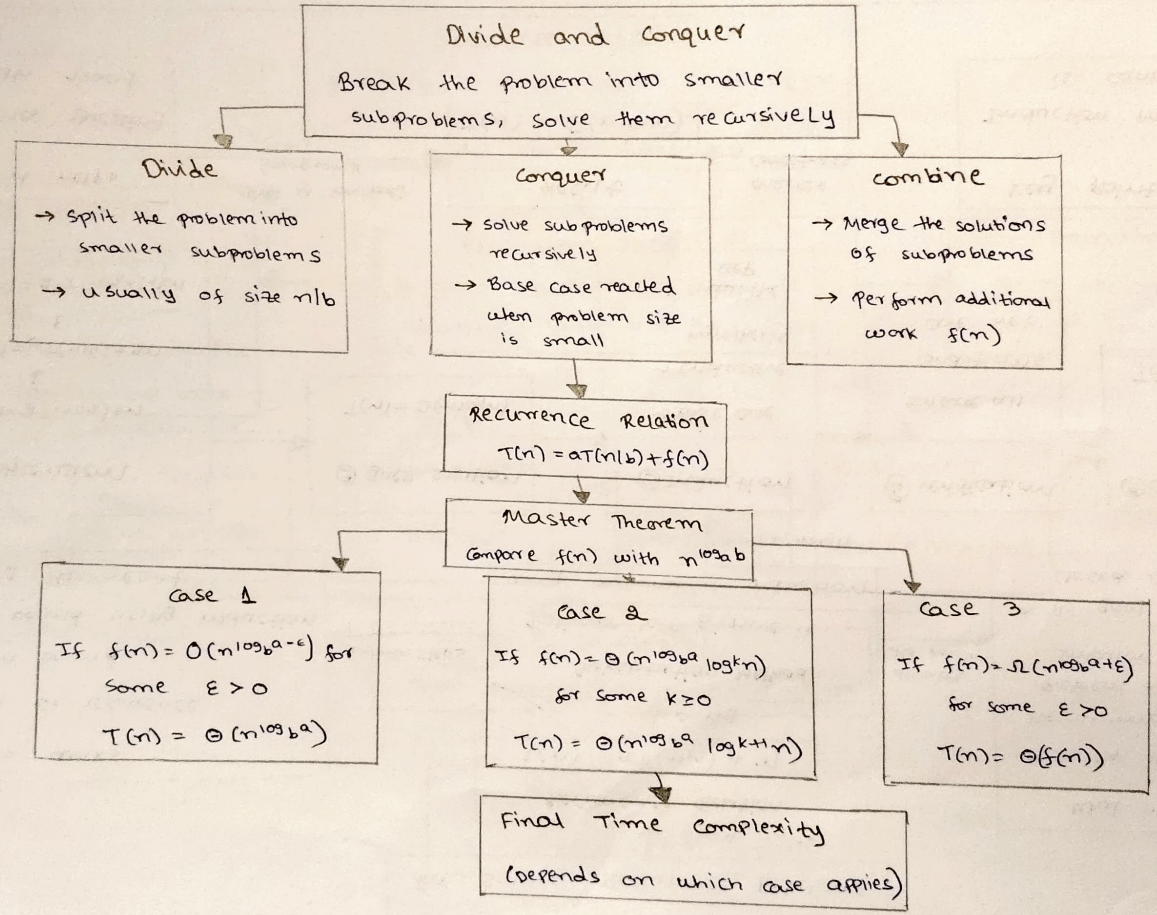
CO1-AT2

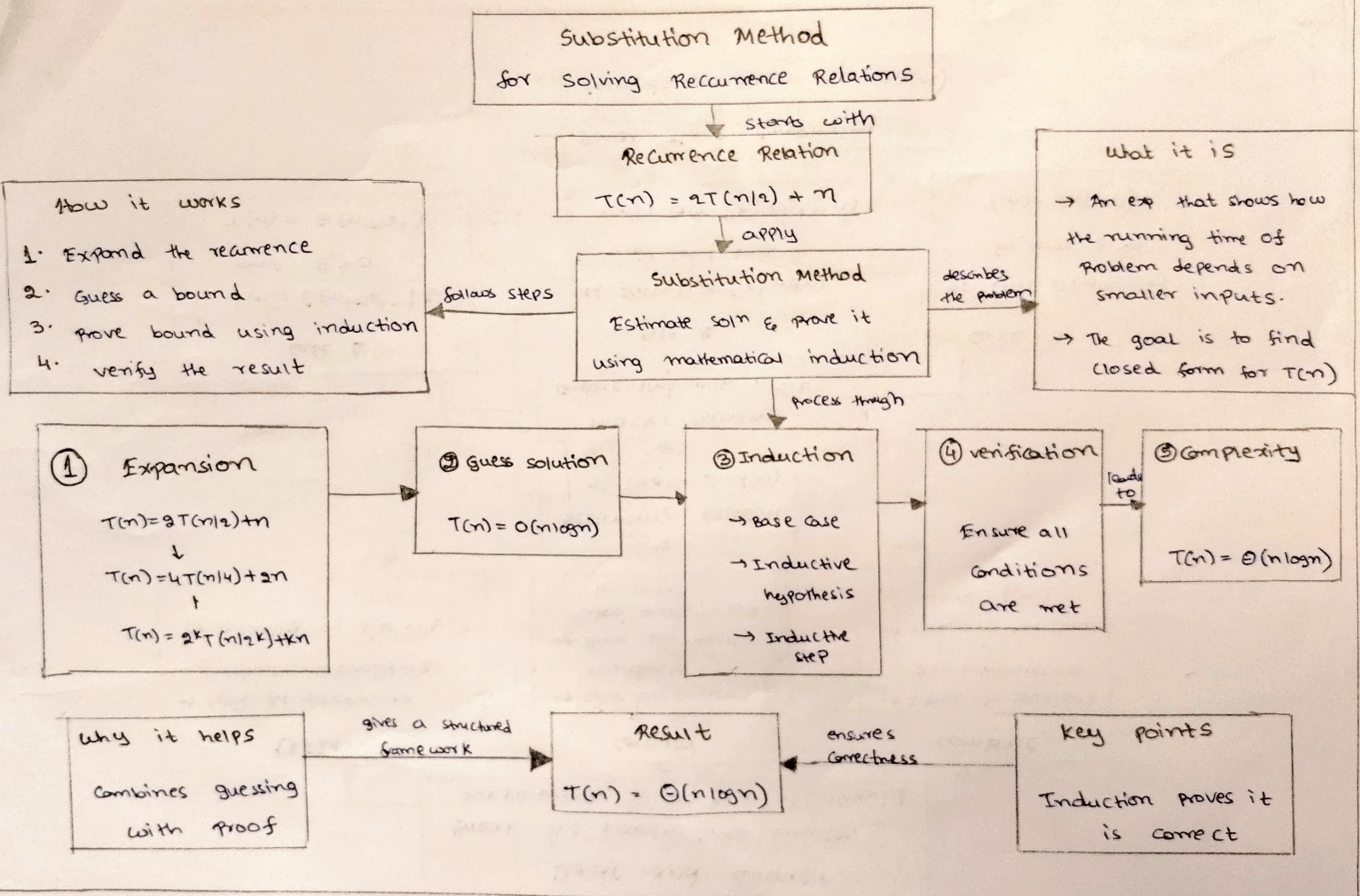
Name :- N. Rujitha Naga Sai Lakshmi
Reg. NO :- 192572119

Course Code :- CSA0620
Date :- 22-10-2026




Algorithm
A finite sequence of instructions to solve a problem
Input Size (n)
No. of elements or amount of input data
Time Complexity
How the running time grows as n increases
Asymptotic Notation
Describes the growth rate of the running time for large values of n
Big-O (O)
→ upper bound
→ shows Max growth
→ O(n²)
Big-Theta (Θ)
→ Thight bound
→ shows exact growth rate
→ Θ(n²)
Big-omega (Ω)
→ Lower bound
→ shows Minimum growth
→ Ω(n²)
Efficiency
Compare growth rates and estimate scalability for large inputs


Divide and Conquer
Break the problem into smaller subproblems, solve them recursively
Divide
→ Split the problem into smaller subproblems
→ usually of size n/b
Conquer
→ solve subproblems recursively
→ Base case reached when problem size is small
combine
→ Merge the solutions of subproblems
→ Perform additional work f(n)
Recurrence Relation
T(n) = aT(n/b) + f(n)
Master Theorem
Compare f(n) with $n^{\log_b a}$
Case 1
If $f(n) = O(n^{\log_b a - \epsilon})$ for some $\epsilon > 0$
$T(n) = \Theta(n^{\log_b a})$
Case 2
If $f(n) = \Theta(n^{\log_b a} \log^k n)$ for some $k \geq 0$
$T(n) = \Theta(n^{\log_b a} \log^{k+1} n)$
Case 3
If $f(n) = \Omega(n^{\log_b a + \epsilon})$ for some $\epsilon > 0$
$T(n) = \Theta(f(n))$
Final Time Complexity
(Depends on which case applies)


Substitution Method
for Solving Recurrence Relations
starts with
Recurrence Relation
T(n) = 2T(n/2) + n
apply
Substitution Method
Estimate soln & prove it using mathematical induction
follows steps
How it works
1. Expand the recurrence
2. Guess a bound
3. Prove bound using induction
4. verify the result
describes the problem
What it is
→ An eqn that shows how the running time of Problem depends on smaller inputs.
→ The goal is to find closed form for T(n)
process through
1 Expansion
T(n)=2T(n/2)+n
T(n)=4T(n/4)+2n
T(n)=2^kT(n/2^k)+kn
2 Guess solution
T(n)=O(nlogn)
3 Induction
→ Base case
→ Inductive hypothesis
→ Inductive step
4 verification
Ensure all conditions are met
leads to
5 Complexity
T(n)=Θ(nlogn)
Why it helps
Combines guessing with proof
gives a structured framework
Result
T(n)=Θ(nlogn)
ensures correctness
Key points
Induction proves it is correct

Recurrence Relation

A relation that defines a function in terms of smaller instances of the same function

Recurrence Relation

$T(n) = aT(n/b) + f(n)$

a - no. of subproblems

n/b - size of each subproblems

$f(n)$ - non-recursive work

Recursion Tree

A Tree structure to visualize recursive calls and their costs

Branching Factor(a)

No. of recursive calls at each node

Level Cost

at each level = $\Theta(n)$

Height of Tree

$h \approx \log_b n$

Total cost

$T(n) = \sum_{i=0}^{h} (\text{cost at level } i)$

Base cases

Cost = $\Theta(1)$ per leaf

Time Complexity

Total running time of the algorithm

Final result

$T(n) = 2T(n/2) + n \rightarrow \Theta(n \log n)$

Algorithm

A sequence of computation steps to solve problem

Operation Count

Count basic operations like comparisions assignments, arithemetic operations, loop

Growth function

Shows how the no. of operations grows as input size (n) increases

Big-O Notation

$T(n) = O(g(n))$

related to

Worst-Case Analysis

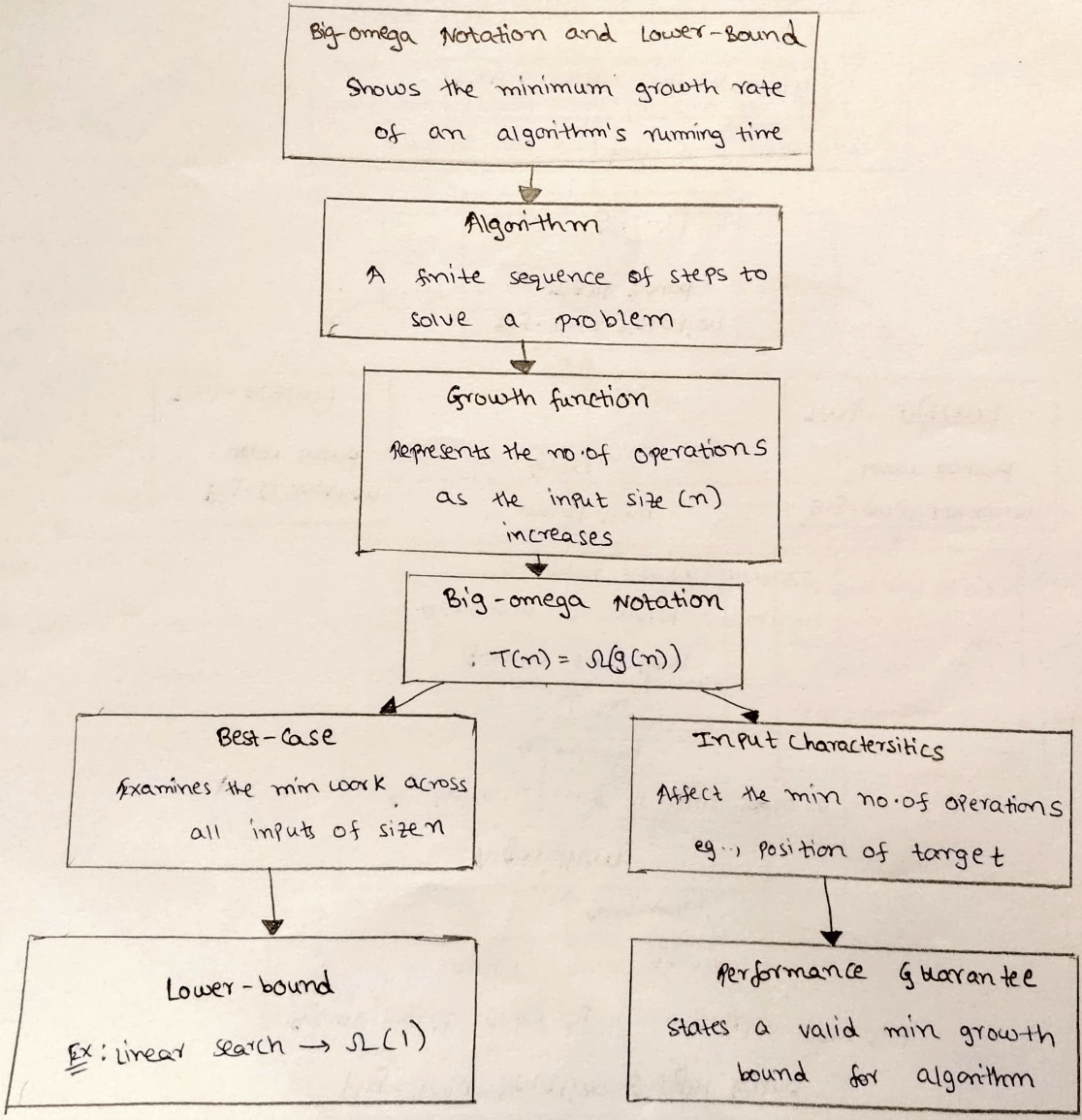
→ Considers the maximum no. of operations over all inputs of size n

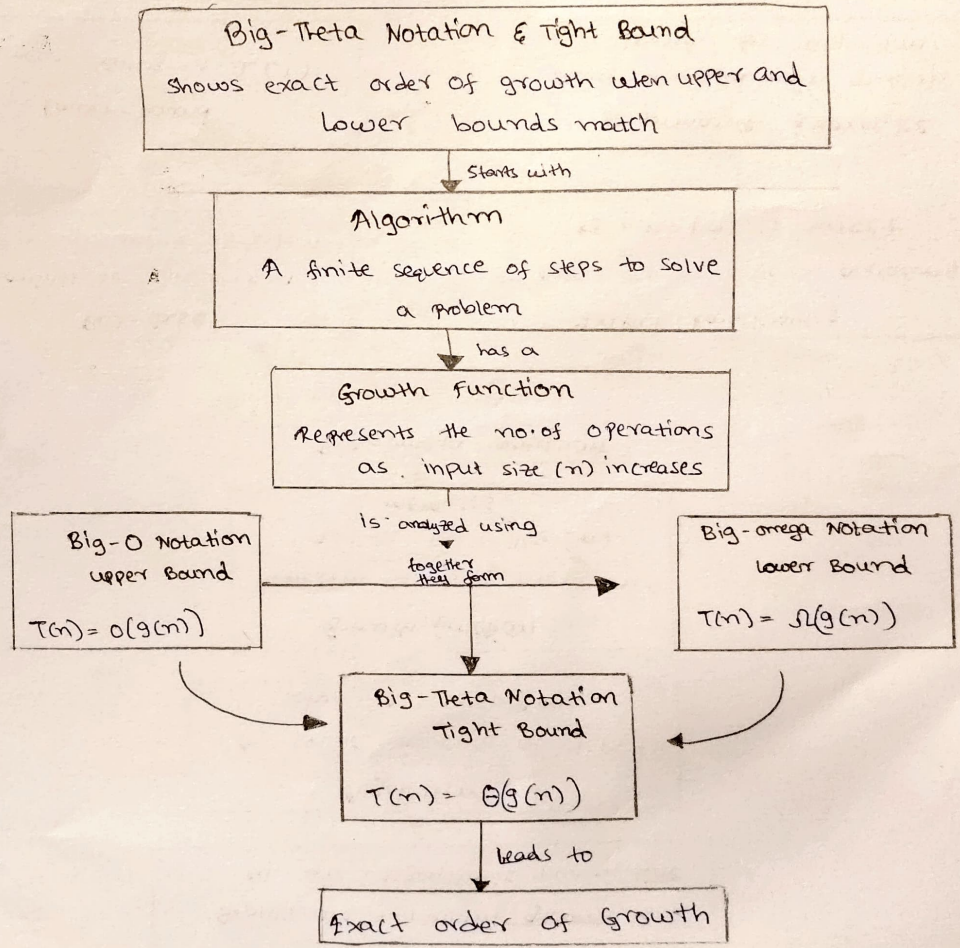
→ Often used with Big-O to give a safe upper limit

Upper Bound

$0 \leq T(n) \leq c \cdot g(n)$

$n \geq n_0$


Big-omega Notation and Lower-Bound
Shows the minimum growth rate of an algorithm's running time
Algorithm
A finite sequence of steps to solve a problem
Growth function
Represents the no. of operations as the input size (n) increases
Big-omega Notation
T(n) = Ω(g(n))
Best-Case
Examines the min work across all inputs of size n
Input Characteristics
Affect the min no. of operations
eg., position of target
Lower-bound
Ex: Linear search → Ω(1)
Performance Guarantee
States a valid min growth bound for algorithm


Big-Theta Notation & Tight Bound
Shows exact order of growth when upper and lower bounds match
Starts with
Algorithm
A finite sequence of steps to solve a problem
has a
Growth Function
Represents the no. of operations as input size (n) increases
Is analyzed using
Big-O Notation
Upper Bound
T(n) = O(g(n))
together they form
Big-omega Notation
Lower Bound
T(n) = Ω(g(n))
Big-Theta Notation
Tight Bound
T(n) = Θ(g(n))
leads to
Exact order of Growth

Input size and different Growth Rates

Shows how changes in input size leads to different growth patterns and how these patterns are classified using asymptotic

Algorithm

A set of steps to solve a problem

Time Complexity

Shows how running time grows with n

Constant Time O(1)

- → Time does not change with n
- → Ex: Accessing element in an array

Logarithmic Time O(log n)

- → Grows very slowly with n
- → Ex: Binary search, Divide and Conquer algorithms

Linear Time O(n)

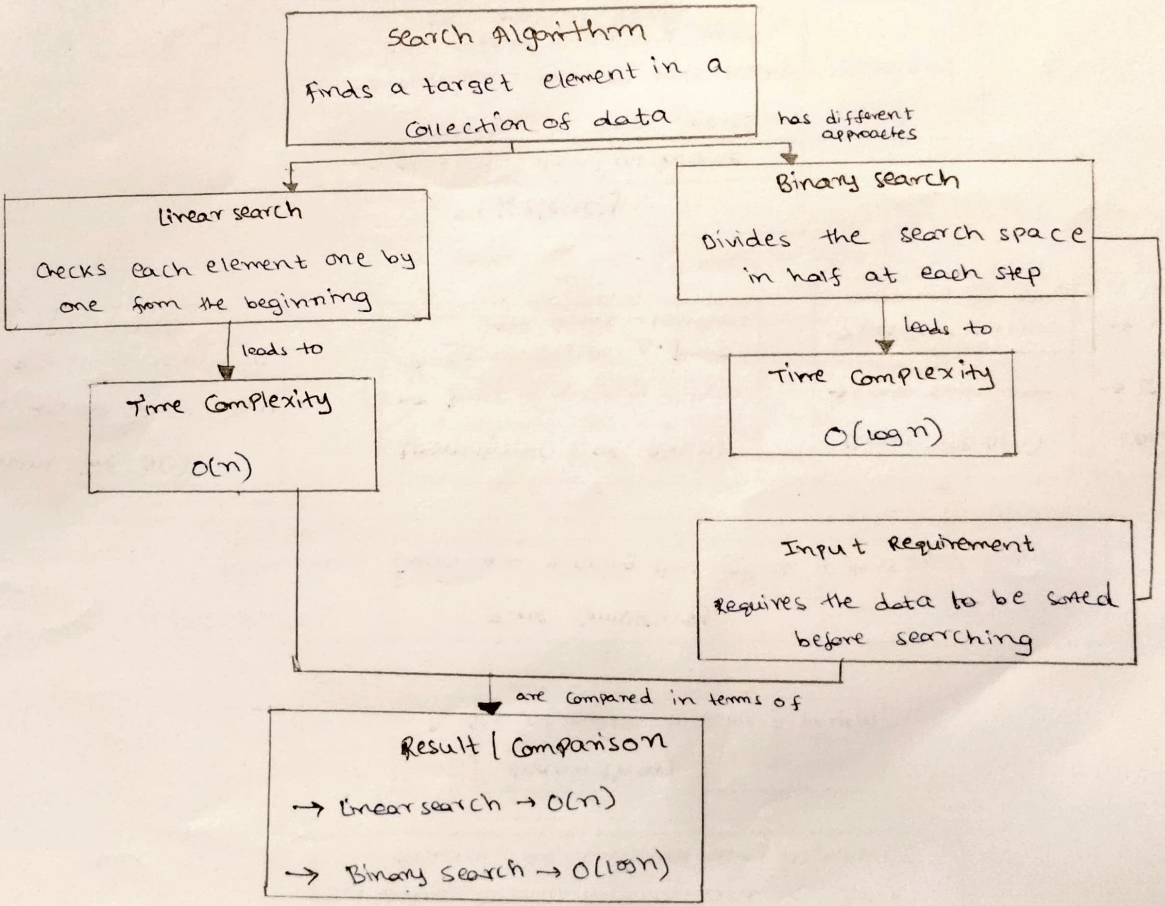
- → Grows proportionally with n
- → Ex: Linear search, Traversing an array

Quadratic Time O(n^2)

- → Grows much faster with n
- → Ex: Nested loops, bubble sort

Efficiency

- → Quadratic algorithms become inefficient for large inputs
- → Asymptotic notation helps classify and compare algorithms based on order of growth


Search Algorithm
finds a target element in a collection of data
has different approaches
Linear search
Checks each element one by one from the beginning
leads to
Time Complexity
O(n)
Binary search
Divides the search space in half at each step
leads to
Time Complexity
O(log n)
Input Requirement
Requires the data to be sorted before searching
are compared in terms of
Result | Comparison
→ Linear search → O(n)
→ Binary search → O(log n)